Project proposal

DiagraML: a programmatic diagramming library for OCaml

Arya Golkari, Trinity College
Computer Science Tripos, Part II
2026–2027

Marking supervisor Day-to-day supervisor	Professor Anil Madhavapeddy
Directors of studies	Dr Sean Holden Professor Neel Krishnaswami Professor Frank Stajano

Contents

1 Introduction and description 2
1.1 Core goals . . . 2
1.2 Extension goals . . . 2
2 Starting point 3
3 Project structure 3
4 Success criteria and evaluation plan 3
5 Work plan 4
6 Resource declaration 4

1 Introduction and description

As a general rule, adding visualisations to texts increases comprehensibility [1]. Data structures in the field of computer science are not an exception to this rule; visualisations can greatly enhance students' learning [2]. However, producing usable and intuitive diagrams is time-consuming, and as a consequence, is often neglected.

Spytial[1] aims to alleviate this problem. It defines a declarative domain-specific language (DSL), capturing spatial features of data, from which interactive diagrams are rendered [3]. I recommend the reader take a moment to familiarise themselves with Spytial via its playground.

1.1 Core goals

The aim of this project is to create a programmatic diagramming library for OCaml, heavily inspired by Spytial. This comprises two primary components: a composable DSL for describing structured data and spatial invariants, and a constraint solver that realises the diagram layout.

The DSL shall be embedded in OCaml, and need not maintain a one-to-one correspondence with existing Spytial DSLs in Python or other languages. This presents an opportunity for reducing the space of compilable yet syntactically invalid diagrams.

The constraint solver shall be an implementation, in OCaml, of Cassowary, an incremental algorithm for efficient linear arithmetic constraint-solving [4]. It is notable that, to the best of my knowledge, this will be the first OCaml implementation.

The output shall be a rendered diagram (ideally, interactive) if the constraints are satisfiable. Otherwise, the program shall produce an error message.

1.2 Extension goals

If time permits, I shall endeavour to embellish the project beyond the core requirements.

Interaction Output a browser target (likely using `js_of_ocaml`) for interactive use. Graph nodes become draggable.

IIS reporting Identify irreducible infeasible subsystems for unsatisfiable constraints and report them. Reports can be textual or visual.

Non-rejection Produce best-effort diagrams when the constraints are unsatisfiable by weakening or dropping some constraints.

Bespoke constraint-solving Devise a bespoke incremental constraint-solving algorithm, tailored to the specific use case of a programmatic diagramming library.

Expression Propose and implement new constraints or directives.

[1] Sometimes stylised as sPyTial.

2 Starting point

I have done no prior work for this project other than the preliminary research required to produce this document.

3 Project structure

In essence, the project comprises four components: the DSL, the intermediate representation, the frontend, and the backend.

DSL A language for describing diagrams and spatial invariants, embedded within OCaml. It will most likely exploit OCaml's generalised algebraic datatypes (GADTs), but the specific design will be decided as part of the project.

Intermediate representation A language for representing diagrams and spatial invariants that is lower-level than the DSL, and is invisible to the user. In Spytial, this is embedded within JSON and YAML. Here, the intermediate representation will (most likely) be in OCaml.

Frontend A compiler for translating from the DSL into the intermediate representation.

Backend A compiler for translating from the intermediate representation into a visualisable output. It comprises constraint-solving. The core requirement is for the output to be a static (likely SVG) file. In Spytial, the backend is called `spytial-core` and is largely written in TypeScript. Here, it will be written primarily or purely in OCaml.

4 Success criteria and evaluation plan

The project will be considered a success if the resulting diagramming library is able to *correctly* produce all 22 examples used for benchmarking in the paper that introduced Spytial [3]. Examples are a representative sample of diagrams in the popular and widely taught textbook on algorithms and data structures, *Introduction to Algorithms (CLRS)* [5]. An outputted diagram is *correct* if all constraints (but not necessarily decorative directives) are obeyed.

Evaluation will be multi-faceted.

Performance I shall measure diagram generation times (excluding rendering times) for the above-mentioned examples and for different diagram sizes, using my implementation and the original Python Spytial implementation, and compare them.

Representability I shall compare, qualitatively, the space of programmatically representable or compilable diagram definitions that are ill-formed and cannot be converted into a rendered visual, between my implementation and the original Spytial implementation.

Aestheticity I shall assess, qualitatively, how the diagrams produced by my OCaml library compare, from an aesthetic perspective, to those produced by Spytial. Aestheticity is positively correlated with user efficiency and engagement [6]. Aestheticity is inherently subjective, but the evaluation will focus on more objective aspects, such as lack of unnecessary kinks in links and consistent spacing between nodes.

5 Work plan

Due date	Duration (days)	Task
2026-10-18	10	Read *Real World OCaml* [7]
2026-10-25	7	Design the DSL and intermediate representation
2026-11-08	14	Implement the frontend
2026-11-15	7	Compose frontend with Spytial's backend and test Milestone frontend complete
2026-11-29	14	Read literature on Cassowary and its implementations
2026-12-13	14	Implement Cassowary in OCaml
2026-12-20	7	Test the Cassowary implementation (and debug) Milestone constraint solver complete
2027-01-03	14	Implement backend (with hard constraints only)
2027-01-17	14	Add weak constraints (for improved visuals)
2027-01-31	14	Create CLRS diagrams in Spytial and OCaml Milestone success criteria met
2027-02-05	5	Deadline write *progress report*
2027-02-10	5	Prepare *progress report presentation*
2027-02-24	14	Extension implement interactive output
2027-03-10	14	Extension improve responsiveness
2027-03-24	14	Extension implement textual IIS reporting
2027-04-04	11	Extension implement visual IIS reporting
2027-04-11	7	Write dissertation chapters 1 and 2
2027-04-18	7	Write dissertation chapter 3
2027-04-25	7	Write dissertation chapter 4
2027-05-02	7	Write dissertation chapter 5
2027-05-14	12	Deadline polish dissertation

6 Resource declaration

I will use my personal laptop, an MSI Raider GE78 HX 14VHG with an Intel Core i9-14900HX processor and 32 GB of RAM running Windows 11 Pro. I accept full responsibility for this machine and I have made contingency plans to protect myself against hardware and/or software failure.

I will also use OCaml, Spytial, Cassowary, and tools within their ecosystems, all of which are open source.

I will use GitHub for version control (and as backup). I will store copies of anything not committed to a remote git repository on a flash drive.

I will use Overleaf to produce the dissertation (and have used it to produce this proposal). I will also store local copies of all LaTeX documents to protect against any service unavailability.

References

[1] M. Tuscher, "Processing speed and comprehensibility of visualizations and texts," *Proceedings of the Central European Seminar on Computer Graphics*, 2022.

[2] T. Suleman, "Visualisation of algorithms and linear data structures: Literature review," University of Cape Town, Tech. Rep., 2024.

[3] S. Prasad, M. Tu, K. Kashyap, T. Nelson, and S. Krishnamurthi, "Diagramming program values by spatial refinement," *Proceedings of the ACM on Programming Languages*, vol. 10, no. PLDI, pp. 705–729, 2026.

[4] G. J. Badros and A. Borning, "The cassowary linear arithmetic constraint solving algorithm: Interface and implementation," University of Washington, Seattle, WA, USA, Tech. Rep. UW-CSE-98-06-04, 1998.

[5] C. E. Leiserson, R. L. Rivest, T. H. Cormen, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 1994.

[6] N. Cawthon and A. Vande Moere, "The effect of aesthetic on the usability of data visualization," in *2007 11th International Conference Information Visualization (IV'07)*, IEEE, 2007, pp. 637–648.

[7] A. Madhavapeddy and Y. Minsky, *Real World OCaml: Functional programming for the masses*. Cambridge University Press, 2022.